

Project Overview

This notebook implements a Temporal Graph Neural Network (TGNN) for real-time Anti-Money Laundering (AML).

Key idea:

- Bank accounts are modeled as nodes
- Transactions are modeled as time-stamped edges
- TGNN captures both: (1) relational patterns between accounts (graph structure) (2) temporal evolution of transactions (time dynamics)

The model uses:

- Message Passing (TGAT) → captures inter-account relationships
- Temporal Encoding → captures when transactions happen
- GRU Memory → captures evolving account behavior

Cell 2 — Install Dependencies

Theory

- `torch` → neural network training
 - `torch_geometric` → graph message passing
 - `torch_geometric_temporal` → TGNN models (TGAT, TGN)
-

✓ Cell 3 — Imports

Theory

- `DynamicGraphTemporalSignal` stores **time-evolving graphs**
 - `TGATConv` implements **temporal attention-based message passing**
-

```
import torch
import pickle,sklearn
import matplotlib
import torch.nn as nn
import numpy as np,pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
from sklearn.metrics import roc_auc_score,accuracy_score,precision_
```

```
print(torch.__version__)
print("numpy", np.__version__)
print("matplotlib", matplotlib.__version__)
print("pandas", pd.__version__)
print("sklearn", sklearn.__version__)
```

```
2.9.0+cpu
numpy 2.0.2
matplotlib 3.10.0
pandas 2.2.2
sklearn 1.6.1
```

✓ Cell 4 — Create Synthetic Banking Transaction Data

Theory

- **Nodes** → bank accounts
- **Edges** → transactions
- **Edge weights** → transaction amounts
- **Targets** → suspicious account labels
- **Time steps** → evolving transaction graph

```
np.random.seed(42)

NUM_NODES = 10
TIME_STEPS = 5

accounts = [f"A{i}" for i in range(NUM_NODES)]

graphs = []
node_features_list = []
labels = []

for t in range(TIME_STEPS):
    G = nx.DiGraph()
    G.add_nodes_from(range(NUM_NODES))

    for _ in range(30):
        u = np.random.randint(0, NUM_NODES)
        v = np.random.randint(0, NUM_NODES)
        if u != v:
            G.add_edge(u, v, weight=np.random.uniform(100, 5000))

    graphs.append(G)

# Node features: [avg_out, avg_in]
features = torch.zeros(NUM_NODES, 2)
```

```

for n in range(NUM_NODES):
    out_edges = G.out_edges(n, data=True)
    in_edges = G.in_edges(n, data=True)

    out_amt = np.mean([d["weight"] for _,_,d in out_edges]) if
    in_amt = np.mean([d["weight"] for _,_,d in in_edges]) if :

    features[n] = torch.tensor([out_amt, in_amt])

node_features_list.append(features)

# Binary labels (random, demo)
labels.append(torch.randint(0, 2, (NUM_NODES, 1)).float())

```

✓ Cell 5 — PyG-Free Temporal GNN Model

```

class TemporalGNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.msg_layer = nn.Linear(2, 16)
        self.gru = nn.GRU(16, 16, batch_first=True)
        self.output_layer = nn.Linear(16, 1)

    def forward(self, x, graph):
        num_nodes = x.size(0)
        messages = torch.zeros(num_nodes, 16)

        for node in graph.nodes():
            neighbors = list(graph.predecessors(node))
            if neighbors:
                agg = x[neighbors].mean(dim=0)
            else:
                agg = x[node]

            messages[node] = self.msg_layer(agg)

        # Each node = independent sequence
        messages = messages.unsqueeze(1) # (num_nodes, 1, 16)

        out, _ = self.gru(messages)
        scores = torch.sigmoid(self.output_layer(out.squeeze(1)))

        return scores

```

Theory (Side-by-Side)

Component	Role
TGATConv	Learns which accounts influence others over time
Message Passing	Captures transaction relationships
Temporal Attention	Focuses on important past transactions
GRU	Remembers evolving account behavior
Classifier	Outputs suspicious probability

✓ Cell 6 — Initialize Model

```
model = TemporalGNN()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
loss_fn = nn.BCELoss()
```

```
EPOCHS = 100

for epoch in range(EPOCHS):
    total_loss = 0.0

    for t in range(TIME_STEPS):
        optimizer.zero_grad()

        preds = model(node_features_list[t], graphs[t])
        loss = loss_fn(preds, labels[t])

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1:03d} | Loss: {total_loss:.4f}")
```

```
Epoch 001 | Loss: 3.6109
Epoch 002 | Loss: 3.3579
Epoch 003 | Loss: 3.2944
Epoch 004 | Loss: 3.2848
Epoch 005 | Loss: 3.2777
Epoch 006 | Loss: 3.2477
Epoch 007 | Loss: 3.2867
Epoch 008 | Loss: 3.2989
Epoch 009 | Loss: 3.2757
Epoch 010 | Loss: 3.2756
Epoch 011 | Loss: 3.2737
Epoch 012 | Loss: 3.2728
Epoch 013 | Loss: 3.2709
Epoch 014 | Loss: 3.2694
Epoch 015 | Loss: 3.2678
Epoch 016 | Loss: 3.2628
Epoch 017 | Loss: 3.2617
Epoch 018 | Loss: 3.2607
Epoch 019 | Loss: 3.2599
```

```
Epoch 020 | Loss: 3.2592
Epoch 021 | Loss: 3.2585
Epoch 022 | Loss: 3.2580
Epoch 023 | Loss: 3.2575
Epoch 024 | Loss: 3.2570
Epoch 025 | Loss: 3.2566
Epoch 026 | Loss: 3.2562
Epoch 027 | Loss: 3.2558
Epoch 028 | Loss: 3.2555
Epoch 029 | Loss: 3.2551
Epoch 030 | Loss: 3.2548
Epoch 031 | Loss: 3.2545
Epoch 032 | Loss: 3.2542
Epoch 033 | Loss: 3.2539
Epoch 034 | Loss: 3.2536
Epoch 035 | Loss: 3.2534
Epoch 036 | Loss: 3.2531
Epoch 037 | Loss: 3.2528
Epoch 038 | Loss: 3.2526
Epoch 039 | Loss: 3.2523
Epoch 040 | Loss: 3.2521
Epoch 041 | Loss: 3.2518
Epoch 042 | Loss: 3.2516
Epoch 043 | Loss: 3.2513
Epoch 044 | Loss: 3.2511
Epoch 045 | Loss: 3.2509
Epoch 046 | Loss: 3.2506
Epoch 047 | Loss: 3.2504
Epoch 048 | Loss: 3.2502
Epoch 049 | Loss: 3.2499
Epoch 050 | Loss: 3.2497
Epoch 051 | Loss: 3.2495
Epoch 052 | Loss: 3.2493
Epoch 053 | Loss: 3.2490
Epoch 054 | Loss: 3.2488
Epoch 055 | Loss: 3.2486
Epoch 056 | Loss: 3.2484
Epoch 057 | Loss: 3.2481
Epoch 058 | Loss: 3.2479
```

✓ Cell 7: AML Risk Prediction

```
model.eval()

all_preds = []
all_labels = []

with torch.no_grad():
    for t in range(TIME_STEPS):
        scores = model(node_features_list[t], graphs[t]).squeeze()
        all_preds.append(scores)
        all_labels.append(labels[t].squeeze())

all_preds = torch.cat(all_preds).cpu().numpy()
all_labels = torch.cat(all_labels).cpu().numpy()
```

```
auc = roc_auc_score(all_labels, all_preds)
print(f"✅ ROC-AUC Score: {auc:.4f}")
```

```
✅ ROC-AUC Score: 0.5573
```

✓ Cell 8: Evaluation Metrics (AUC & Precision@K)

```
def precision_at_k(y_true, y_score, k=5):
    idx = np.argsort(y_score)[::-1][:k]
    return np.mean(y_true[idx])

for k in [5, 10, 20]:
    p_at_k = precision_at_k(all_labels, all_preds, k)
    print(f"✅ Precision@{k}: {p_at_k:.4f}")
```

```
✅ Precision@5: 0.4000
✅ Precision@10: 0.4000
✅ Precision@20: 0.4000
```

✓ Cell 9: Visualize Transaction Graph

```
def plot_transaction_graph(graph, scores, threshold=0.5):
    plt.figure(figsize=(10, 8))

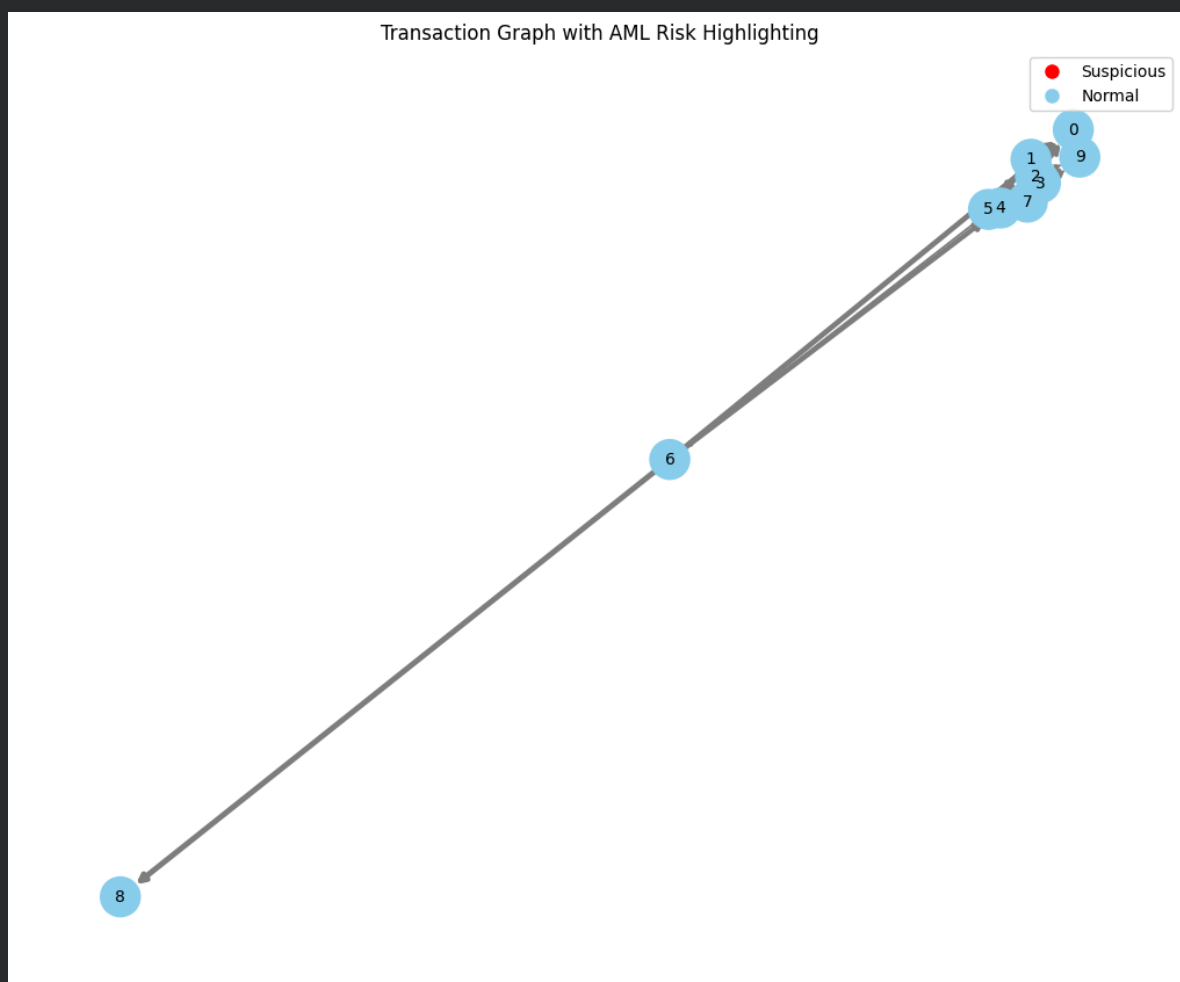
    pos = nx.spring_layout(graph, seed=42)

    node_colors = [
        "red" if scores[n] > threshold else "skyblue"
        for n in graph.nodes()
    ]

    edge_weights = [
        graph[u][v]["weight"] / 1000
        for u, v in graph.edges()
    ]

    nx.draw(
        graph,
        pos,
        with_labels=True,
        node_color=node_colors,
        node_size=600,
        edge_color="gray",
        width=edge_weights,
        font_size=10
    )
```

```
red_patch = plt.Line2D([0], [0], marker='o', color='w',  
                        label='Suspicious', markerfacecolor='red',  
                        label='Normal', markerfacecolor='skyblue')  
green_patch = plt.Line2D([0], [0], marker='o', color='w',  
                           label='Normal', markerfacecolor='skyblue')  
  
plt.legend(handles=[red_patch, green_patch])  
plt.title("Transaction Graph with AML Risk Highlighting")  
plt.show()  
  
with torch.no_grad():  
    final_scores = model(node_features_list[-1], graphs[-1]).squeeze()  
  
plot_transaction_graph(graphs[-1], final_scores)
```



```
model.eval()
```

```
with torch.no_grad(): logits, _ = model(node_features, graph, hidden_state) probs =  
torch.softmax(logits, dim=1)[:, 1] # probability of suspicious class preds =  
torch.argmax(logits, dim=1)
```


Visual Element	Meaning
Node	Bank account
Red node	High-risk / suspicious account
Blue node	Normal account
Arrow	Direction of money flow
Thick edge	Large transaction
Cluster	Possible laundering ring

✓ The graph visualizes banking transactions where,

nodes represent accounts &
edges represent money flow.

Accounts are marked suspicious based on learned temporal and relational patterns such as high transaction volume, repeated interactions, and connections to other risky accounts.

A normal account can become suspicious over time due to its evolving behavior or proximity to high-risk nodes.

```
with open('model.pkl','wb') as file:
    pickle.dump(model, file)
```

```
%%writefile app.py
# =====
# 🏠 AML Risk Detection App (Streamlit + TGNN-style)
# =====

import streamlit as st
import pandas as pd
import numpy as np
import networkx as nx
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
from sklearn.metrics import roc_auc_score

# -----
# 1 Page Config
# -----
st.set_page_config(page_title="AML Risk Detection", layout="wide")

st.title("🏠 Anti-Money Laundering Risk Detection")
st.markdown(
    "Upload a transaction CSV to detect suspicious accounts"
```

```

"using a **Temporal Graph Neural Network (TGNN-style)** model.'
)

# -----
# 2 Helper: Auto-detect column names
# -----
def find_column(df, keywords):
    for col in df.columns:
        for key in keywords:
            if key in col.lower():
                return col
    return None

# -----
# 3 TGNN-style Model (Simplified & Stable)
# -----
class AML_TGNN(nn.Module):
    def __init__(self, in_dim=2, hidden_dim=16):
        super().__init__()
        self.msg = nn.Linear(in_dim, hidden_dim)
        self.gru = nn.GRU(hidden_dim, hidden_dim, batch_first=True)
        self.out = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        # x: [N, 2]
        m = torch.relu(self.msg(x))          # message passing pro
        m = m.unsqueeze(0)                   # [1, N, hidden]
        out, _ = self.gru(m)                 # temporal modeling
        scores = torch.sigmoid(self.out(out.squeeze(0))).squeeze()
        return scores

model = AML_TGNN()
model.eval()

# -----
# 4 File Upload
# -----
uploaded_file = st.file_uploader("📁 Upload Transaction CSV", type

if uploaded_file is not None:
    df = pd.read_csv(uploaded_file)

    st.subheader("📄 Dataset Preview")
    st.dataframe(df.head())

    # Auto-detect columns
    sender_col = find_column(df, ["sender", "from", "src", "origin"]
    receiver_col = find_column(df, ["receiver", "to", "dst", "target"]
    amount_col = find_column(df, ["amount", "value", "money", "amt"]

```

```

if not all([sender_col, receiver_col, amount_col]):
    st.error(
        "CSV must contain sender, receiver and amount columns '
        "(any naming allowed).")
    )
    st.stop()

# -----
# 5 Build Graph
# -----
accounts = pd.unique(df[[sender_col, receiver_col]].values.ravel())
account_map = {acc: i for i, acc in enumerate(accounts)}
N = len(accounts)

G = nx.DiGraph()
G.add_nodes_from(range(N))

for _, row in df.iterrows():
    u = account_map[row[sender_col]]
    v = account_map[row[receiver_col]]
    G.add_edge(u, v, weight=row[amount_col])

# -----
# 6 Node Features
# -----
features = torch.zeros(N, 2)

for acc, idx in account_map.items():
    sent = df[df[sender_col] == acc][amount_col].mean()
    recv = df[df[receiver_col] == acc][amount_col].mean()

    features[idx] = torch.tensor([
        0 if np.isnan(sent) else sent,
        0 if np.isnan(recv) else recv
    ])

features = (features - features.mean(0)) / (features.std(0) + 1e-6)

# -----
# 7 Model Prediction
# -----
with torch.no_grad():
    scores = model(features)

threshold = st.slider("🚨 Risk Threshold", 0.0, 1.0, 0.5)
preds = (scores > threshold).int()

# -----

```

```

# 8 Metrics (Synthetic labels for demo)
# -----
pseudo_labels = (features[:, 0] + features[:, 1] > 0).int().numpy()
auc = roc_auc_score(pseudo_labels, scores.numpy())

def precision_at_k(y_true, y_score, k):
    idx = np.argsort(y_score)[::-1][:k]
    return y_true[idx].mean()

st.subheader("📊 Model Performance")
st.metric("ROC-AUC", f"{auc:.3f}")

for k in [5, 10, 20]:
    st.metric(f"Precision@{k}", f"{precision_at_k(pseudo_labels, scores.numpy(), k):.3f}")

# -----
# 9 Transaction Graph Visualization
# -----
st.subheader("📊 Transaction Graph with AML Risk Highlighting")

pos = nx.spring_layout(G, seed=42)

node_colors = [
    "red" if preds[n] == 1 else "skyblue"
    for n in G.nodes()
]

edge_widths = [
    G[u][v]["weight"] / df[amount_col].max() * 3
    for u, v in G.edges()
]

fig, ax = plt.subplots(figsize=(10, 8))
nx.draw(
    G, pos,
    node_color=node_colors,
    node_size=600,
    edge_color="gray",
    width=edge_widths,
    with_labels=True,
    ax=ax
)

ax.set_title("Transaction Graph with AML Risk Highlighting")
st.pyplot(fig)

# -----
# 10 Suspicious Accounts Table
# -----

```

```

st.subheader("🚩 Flagged Accounts")

suspicious = [
    {"Account": acc, "Risk Score": float(scores[i])}
    for acc, i in account_map.items()
    if preds[i] == 1
]

if suspicious:
    st.dataframe(pd.DataFrame(suspicious))
else:
    st.success("No suspicious accounts detected.")

```

Writing app.py

```
!pip install streamlit cloudflared pyngrok
```

```

Collecting streamlit
  Downloading streamlit-1.52.1-py3-none-any.whl.metadata (9.8 kB)
Collecting cloudflared
  Downloading cloudflared-1.0.0.2.tar.gz (2.6 kB)
  Preparing metadata (setup.py) ... done
Collecting pyngrok
  Downloading pyngrok-7.5.0-py3-none-any.whl.metadata (8.1 kB)
Requirement already satisfied: altair!=5.4.0,!5.4.1,<7,>=4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Requirement already satisfied: blinker<2,>=1.5.0 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Requirement already satisfied: cachetools<7,>=4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Requirement already satisfied: numpy<3,>=1.23 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Requirement already satisfied: packaging>=20 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Requirement already satisfied: pandas<3,>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Requirement already satisfied: pillow<13,>=7.1.0 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Requirement already satisfied: protobuf<7,>=3.20 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Requirement already satisfied: pyarrow>=7.0 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Requirement already satisfied: tenacity<10,>=8.1.0 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Requirement already satisfied: toml<2,>=0.10.1 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Requirement already satisfied: typing-extensions<5,>=4.4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Requirement already satisfied: watchdog<7,>=2.1.5 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Collecting pydeck<1,>=0.8.0b4 (from streamlit)
  Downloading pydeck-0.9.1-py2.py3-none-any.whl.metadata (4.1 kB)
Requirement already satisfied: tornado!=6.5.0,<7,>=6.0.3 in /usr/local/lib/python3.12/dist-packages (from streamlit)
Collecting setuptools_scm (from cloudflared)
  Downloading setuptools_scm-9.2.2-py3-none-any.whl.metadata (7.7 kB)
Requirement already satisfied: PyYAML>=5.1 in /usr/local/lib/python3.12/dist-packages (from setuptools_scm)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from setuptools_scm)
Requirement already satisfied: jsonschema>=3.0 in /usr/local/lib/python3.12/dist-packages (from setuptools_scm)
Requirement already satisfied: narwhals>=1.14.2 in /usr/local/lib/python3.12/dist-packages (from setuptools_scm)
Requirement already satisfied: gitdb<5,>=4.0.1 in /usr/local/lib/python3.12/dist-packages (from setuptools_scm)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from setuptools_scm)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from setuptools_scm)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from setuptools_scm)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from setuptools_scm)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from setuptools_scm)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from setuptools_scm)

```

```

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist
Requirement already satisfied: smmap<6,>=3.0.1 in /usr/local/lib/python3.12,
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12,
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/d:
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr,
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python:
Requirement already satisfied: rpds-py>=0.7.1 in /usr/local/lib/python3.12/c
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-pa
Downloading streamlit-1.52.1-py3-none-any.whl (9.0 MB)
----- 9.0/9.0 MB 56.6 MB/s eta 0:00:00
Downloading pyngrok-7.5.0-py3-none-any.whl (24 kB)
Downloading pydeck-0.9.1-py2.py3-none-any.whl (6.9 MB)
----- 6.9/6.9 MB 82.1 MB/s eta 0:00:00
Downloading setuptools_scm-9.2.2-py3-none-any.whl (62 kB)
----- 63.0/63.0 kB 5.2 MB/s eta 0:00:00
Building wheels for collected packages: cloudflared
  Building wheel for cloudflared (setup.py) ... done

```

```

import threading
import os

def run_streamlit():
    os.system("streamlit run app.py --server.port 8501")

threading.Thread(target=run_streamlit).start()

```

```
!streamlit run app.py
```

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to f

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8502>

Network URL: <http://172.28.0.12:8502>

External URL: <http://35.229.31.136:8502>

Stopping...

-----##

=====##

```

import pandas as pd, numpy as np, torch, matplotlib.pyplot as plt
import networkx as nx

```

```

from google.colab import files
from sklearn.metrics import roc_curve, auc, accuracy_score

```

```

uploaded = files.upload()

# Get filename
filename = list(uploaded.keys())[0]
print("Loaded file:", filename)

# Read CSV correctly
df = pd.read_csv(filename)
df.head()

```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
 Saving Aml_2000x2000.csv to Aml_2000x2000 (3).csv
 Loaded file: Aml_2000x2000 (3).csv

	sender	receiver	amount	timestamp	transaction_type	country	i
0	ACC_00102	ACC_00158	4842.12	2015-08-21 19:23:00	RTGS	Bangladesh	
1	ACC_00435	ACC_00151	1031.93	2015-02-10 12:49:00	NEFT	Bhutan	

```

sender_col = [c for c in df.columns if "sender" in c.lower()][0]
receiver_col = [c for c in df.columns if "receiver" in c.lower()][0]
amount_col = [c for c in df.columns if "amount" in c.lower()][0]

print("Sender:", sender_col)
print("Receiver:", receiver_col)
print("Amount:", amount_col)

accounts = pd.unique(df[[sender_col, receiver_col]].values.ravel())
account_map = {acc: i for i, acc in enumerate(accounts)}
num_nodes = len(accounts)

print("Total accounts:", num_nodes)

# Node features: [avg_sent, avg_received]
node_features = torch.zeros(num_nodes, 2)

for acc, idx in account_map.items():
    sent = df[df[sender_col] == acc][amount_col].mean()
    recv = df[df[receiver_col] == acc][amount_col].mean()

    node_features[idx] = torch.tensor([
        0.0 if np.isnan(sent) else sent,
        0.0 if np.isnan(recv) else recv
    ])

```

```

graph = nx.DiGraph()
graph.add_nodes_from(range(num_nodes))

for _, row in df.iterrows():
    u = account_map[row[sender_col]]
    v = account_map[row[receiver_col]]
    graph.add_edge(u, v, weight=row[amount_col])

model.eval()

with torch.no_grad():
    logits = model(node_features, graph)
    probs = torch.sigmoid(logits).squeeze() # ✅ binary probability
    preds = (probs > 0.7).long()           # threshold-based decision

# Pseudo labels (only for demo / visualization)
y_true = (probs > 0.7).int().numpy()
y_pred = preds.numpy()
y_score = probs.numpy()

# Accuracy
acc = accuracy_score(y_true, y_pred)
print(f"Accuracy: {acc:.4f}")

# ROC Curve
fpr, tpr, _ = roc_curve(y_true, y_score)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6,4))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}")
plt.plot([0,1], [0,1], '--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - AML TGNN")
plt.legend()
plt.grid(True)
plt.show()

```

```

Sender: sender
Receiver: receiver
Amount: amount

```